

DSA : [Data Structure Algorithm]

1. What is DSA :-

It is the way of organized the data in proper manner.

- To reduced the complexity we use DSA.
- Linked list (single and double) has own properties.

Stack	}	Advantages Disadvantages
Queue		
Tree		
Hashing		

- It not only reduced the complexity. It give and will help find the efficient solution.

2. Linked list :-

Linked list will allow to store the value's non-continuous memory allocation.

In first step, we need to store the value.

```
int num = 10;
```

- If in case we need to store multiple elements that's why we go for arrays.

To store more than one value (static)

```
int array[5] = {0, 1, 3, 4, 5};
```

0	1	2	3	4
0	1	3	4	5
1000	1004	1008	1012	1016

- Static (we can't change because size is fixed).
- Then we go for dynamic memory allocation.

```
int *ptr = malloc(20);
```

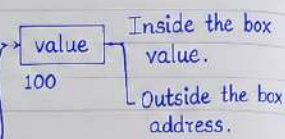
3. Dynamic memory allocation using linked list :-

```
int main()
```

```
{  
    struct node *head = NULL;  
    head = insert data (10);  
    head = insert data (20);  
}
```

```
insert data (int data)
```

```
{  
    struct node *ptr = malloc (size of (struct node));  
    ptr → node value = data;  
    ptr → link = address NULL;  
}
```



4. Advantages :-

```
malloc ( )
```



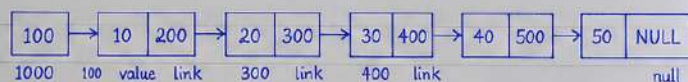
Extended / reduced the memory allocation in runtime.

5. Disadvantages :-

- It will always books into continuous memory allocate.
- We are store the large amount of data.

Linked list they are two types :-

1. node → It will have the value & address of next node.
2. head → It store the address of the first node.



```
struct node {
```

```
    int value;
```

```
    struct node *link → self references pointer.
```

```
};
```

1 time memory allocation. → 1 value & address

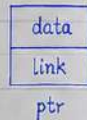
6. Basic Operations on Linked List :-

1. Insertion at first :-

- Create the new node.
- Check if the list is empty.
- If (head == NULL) update the head.
- Otherwise, validate the value and link the new node.

```
void insert_first(int data)
{
    struct node *ptr = malloc(sizeof(struct node));
    ptr->data = data;
    ptr->link = head;
    head = ptr;
}
```

New node structure:-

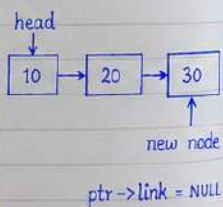


2. Insertion at last :-

- Create the new node.
- Check if the list is empty.
- If (head == NULL) update the head.
- Otherwise, traverse to the last node and link.

```
void insert_last(int data)
{
    struct node *ptr = malloc(sizeof(struct node));
    ptr->data = data;
    ptr->link = NULL;
    if (head == NULL)
        head = ptr;
    else
    {
        struct node *temp = head;
        while (temp->link != NULL)
            temp = temp->link;
        temp->link = ptr;
    }
}
```

After inserting at last :-



3. Insertion at a given position (general form) :-

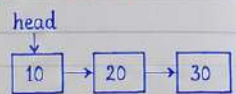
- Traverse to the position (index - 1).
- Link the new node in between.
- Handle the boundary cases (empty list, invalid position).

4. Deletion at first :-

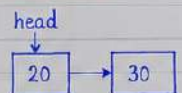
- Check if the list is empty.
- If not empty, delete the first node.
- Update the head to the next node.

```
void delete_first()
{
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    struct node *temp = head;
    head = head->link;
    free(temp);
}
```

Before deletion :-



After deletion :-



5. Deletion at last :-

- Check if the list is empty.
- If only one node is present, free the node and set head = NULL.
- Otherwise, traverse to the second last node, free the last node and set link of second last node to NULL.

6. Searching a value :-

- Traverse the list from the beginning.
- Compare each node's data with the given value.
- If found, return the position / address.
- Otherwise, return -1 (not found).

7. Complete program structure :-

```
int main()
{
    struct node *head = NULL;
    insert_first(10);
    insert_last(20);
    insert_last(30);

    // other operations

    return 0;
}
```

Node structure :-

```
struct node
{
    int data;
    struct node *link;
};
```

Note :-

- Always check for NULL.
- Allocate memory using malloc().
- Free the memory using free().
- Handle all boundary conditions.

Make files :-

How to create make files :-

Name of makefiles → 1. makefile (all small letters)
(or) 2. Makefiles } There are no extensions.

* make file approach bottom to top approach.

Script :-

target : dependencies
<tab> : target recepies
<subtargets>: dependencies
<tab> : recepies

Example :-

```
a.out : main.o add.o sub.o
<tab> gcc main.o add.o sub.o
main.o : main.c
<tab> gcc -c main.c
add.o : add.c
<tab> gcc -c add.c
sub.o : sub.c
<tab> gcc -c sub.c
```

1. check the directory.
2. and go to the makefiles.

How to run :-

make

→ It will execute the rules and create the files.

Compilation stages :-

Key one + 4 stages are present.

1. Preprocessor stage → handle #include, #define, macros.
2. Macro expansion → comments removed.
3. Compiler → check the syntax and same data type.
4. Assembler → into binary (.s → .o)
5. Linker stage → if all null link the object file and the all libraries and create the file.

Time complexity :- efficiency of program.

They are two types :-

1. Time complexity → consumption.
2. Space complexity → efficiency of program.

They are 3 types of time complexity :-

1. Big Omega (Best case)
2. Big theta (Average case)
3. Big O (Worst case)

a.out my 846B 7.9GB } If it is more consumption of memory, efficiency is data stored are not.

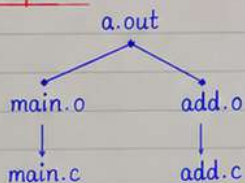
↓
Approximate calculation

Why we going approach ?

Why approach bottom to top ?

- Make only the required files.
- Avoids unnecessary recompilation.
- Handles dependencies automatically.
- Saves time and effort.
- Only modified files will be recompiled.
- Useful for large projects.

Example :-



If main.c is modified, only main.o and a.out will be recompiled. Other files will not be recompiled.

Time complexity example :-

```
for (i = 0; i < n; i++)
{
    printf("HELLO");
}
```

Time complexity = $O(n)$
(because loop runs n times)

$n = 8 \rightarrow 8$ times
 $n = 10 \rightarrow 10$ times
 $n = 16 \rightarrow 16$ times
So, time complexity is directly proportional to n.

Nested loops :-

```
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("HELLO");
    }
}
```

Time complexity = $O(n^2)$

Different loops :-

1. Linear loop → $O(n)$

```
for (i = 0; i < n; i++)
    printf("HELLO");
```

2. Non linear loop (e.g., $i = i * 2$)

```
for (i = 1; i <= n; i = i * 2)
    printf("HELLO");
```

Time complexity = $O(\log n)$

$n = 8 \rightarrow 3$ times (1, 2, 4, 8)

$n = 10 \rightarrow 4$ times (1, 2, 4, 8)

$n = 16 \rightarrow 5$ times (1, 2, 4, 8, 16)

Conclusion :-

- If we reduce the number of operations, we get a better solution.
- Time and space complexity helps to analyze the efficiency.
- Big O represents the worst case, Big Ω best case and Big Θ average case.

1. Loops and Time Complexity :-

```
for (i = 0 ; i < n ; i++)
{
    printf("HELLO");
}
```

One is linear $O(n)$
Time complexity = $O(n)$

```
for (i = 0 ; i < n ; i++)
{
    for (j = 0 ; j < n ; j++)
    {
        printf("HELLO");
    }
}
```

One is nonlinear $O(n^2)$
Time complexity = $O(n^2)$

→ One loop is linear $O(n)$ and nested loop is $O(n^2)$.

2. Sequential manner :-

→ Sequential manner is called linear.

```
int foo(int n)
{
    if (n > 0)
        foo(n - 1);
}
```

Time complexity = $O(n)$

$n = 3$

$3 > 0$	✓
$2 > 0$	✓
$1 > 0$	✓
$0 > 0$	

each call $< O(1)$
Total = $O(n)$

3. Compilation and Execution :-

- OBJ → It is a variable. It will store all the .o files.
- gcc -o <output> <object files> → creates executable file.
- a.out → default executable file name.
- root → root %;
- .o → object files (generated after compilation).
- .c → final executable file.
- .c → source file, will be converted to .o (object file).
- \$(wildcard) → It will bring all the .c files from the current directory.
- \$(patsubst %.c, %.o, \$(wildcard *.c)) → converts .c files to .o files.
- \$@ → It is keyword, will take the name of our target.
- \$^ → It will bring all the dependencies of the source file.

4. Optimization code for Makefile :-

OBJ = \$(patsubst %.c, %.o, \$(wildcard *.c))

all : \$(OBJ)

gcc -o \$@ \$^

clean :

rm -f *.o *.exe

\$(wildcard) → all .c files
\$(patsubst) → convert .c to .o
\$@ → target name
\$^ → all dependencies

5. Make files :-

How to create make files :-

Name of makefiles → 1. makefile (all small letters)
(or)
2. Makefile (capital M) } There are no extensions.

* Makefile approach : bottom to top approach.

Script :-

target : dependencies
<tab> : target recipes
<subtargets> : dependencies
<tab> : recipes (commands)

Example :-

```
a.out : main.o add.o sub.o
<tab> gcc main.o add.o sub.o
main.o : main.c
<tab> gcc -c main.c
add.o : add.c
<tab> gcc -c add.c
sub.o : sub.c
<tab> gcc -c sub.c
```

1. Check the directory.

2. And go to the makefiles.

How to run :-

make

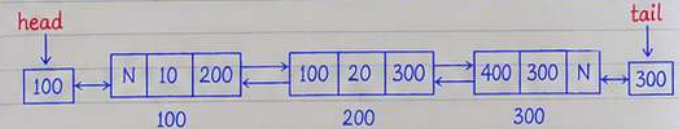
→ It will execute the rules and create the files.

Why bottom to top approach ?

- Make only the required files.
- Avoids unnecessary recompilation.
- Handles dependencies automatically.
- Saves time and effort.
- Only modified files will be recompiled.
- Useful for large projects.

6. Double linked list :-

It is used to move both sides : forward and backward.



7. Stack :-

- LIFO (Last in Last out).
- Stack is approach : bottom to top.
- When repeated all the functions then stack is created.

Stack operations :-

1. Stack creation
2. Push (Add the element)
3. Pop (Delete the element)
4. Peek (even recently added element)
5. Peep (Display / print all elements)

Stack structure :-

```
typedef struct stack
{
    int top;
    int capacity;
    int *array;
}
```

Push operation :-

```
push(stack_t *st, int data)
{
    if (st->top == (st->capacity - 1))
        return failure;
    st->array[++(st->top)] = data;
    return success;
}
```

Pop operation :-

```
pop(stack_t *st)
{
    if (st->top == -1)
        return failure;
    --(st->top);
    return success;
}
```

Advantages :-

→ Both sides we need more.

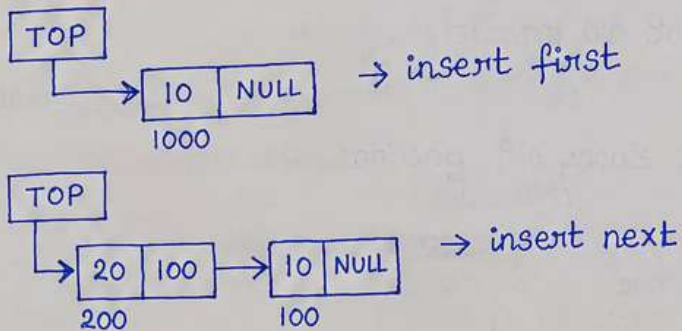
Disadvantage :-

→ More memory consumption is happen.

Create stack :-

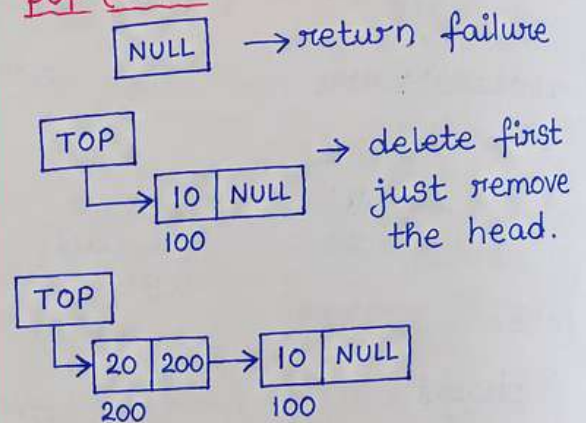
```
typedef struct {  
    int data;  
    glist * stack = NULL;  
};
```

push (stack **top, int data)



TOP → It will hold the address recently added node.

pop (stack_t **top)



Time complexity :-

	SLL	DLL	stack array	stack linked list
insert first	→ O(1)	→ O(1)	push → O(1)	→ O(1)
insert last	→ O(n)	→ O(1)	pop → O(1)	→ O(1)
delete first	→ O(1)	→ O(1)	peek → O(1)	→ O(1)
delete last	→ O(n)	→ O(1)		

Queue :-

{ Linear queue
Circular queue

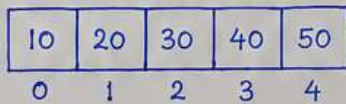
They are two properties.

1. LILO (Last In Last Out)
2. FIFO (First In First Out)

Basic operation :-

1. Enqueue (Add the element)
2. Dequeue (Delete the element)
3. Print Queue

Queue using array :-



If queue is Empty pointing to the -1

front → It will point to the first element
rear → It will point to the recently added element

If (rear == size - 1) {
 printf("Queue is full");
} else {
 add the data in array.

index 0-9 is available, but linear queue don't allow the linear. Shifting is happen. → O(n) will overweigh the data with newdata.

Searching techniques :-

to search the element. They are two types :-

1. Linear search
2. Binary search.

Linear search :- Time complexity = O(n)

Binary search :- {
 O(1) Best case
 O(log n) Average case
 O(log n) Worst case.
 → Used sorted array.

Circular queue using array :-

Basic operation :-

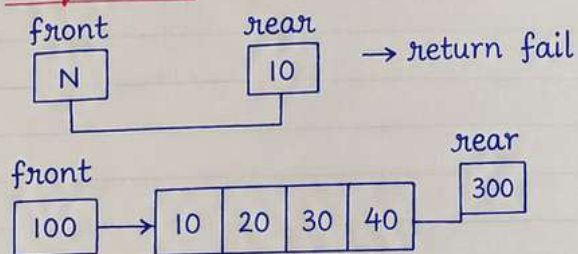
1. Enqueue
2. Dequeue
3. Print queue

```
typedef struct que {  
    int front;  
    int rear;  
    int *arr[100];  
    int count;
```

```
Enqueue(queue_t *q, int data)  
{  
    if (queue_full() == FAILURE)  
        return FAILURE;  
    if (front == -1)  
        front = 0;  
    rear = (rear + 1) % capacity;  
    array[rear % capacity] = data;  
    if (count == capacity)  
        return FAILURE;  
    return SUCCESS;  
}
```

```
Dequeue()  
{  
    if (queue_empty(count == capacity) == FAILURE)  
        return FAILURE;  
    front = (++front) % capacity;  
    count--;  
    return SUCCESS;  
}
```

Dequeue()



Searching techniques :-

To search the element is present or not.

They are two types :-

1. Linear search
2. Binary search

Linear search :- $O(n)$

10	20	30	40	50	60
----	----	----	----	----	----

size = 5

In this, we use the target then the target is search then we findout the data is present are not.

Binary search :- $O(1)$ Best case.

10	20	30	40
----	----	----	----

target is = 30

arr[mid] = target



Then search in the left & right subarray.

eg:- L m h size=5, target=6

10	20	30	40	50
----	----	----	----	----

while (low <= high)

mid = (low + high) / 2

If (target < arr[mid])

search the left subarray.

high = mid - 1;

else if (target > arr[mid])

search the right subarray

low = mid + 1;

else

not found;

Merge sort :-

merge sort (left, mid);
merge sort (right, mid);

merge_compare (arr, size; left, mid,
right, size-mid);

merge sort (int *arr, int size;
int *left, int *right, int size)

```
{
    int i = 0, j = 0, k = 0;
    while (i < size && j < size)
    {
        if (left[i] > right[j])
            arr[k++] = right[j++];
        else
            arr[k++] = left[i++];
    }
    while (j < size)
        arr[k++] = right[j++];
    // copying remaining element of left-
    while (i < size)
        arr[k++] = left[i++];
}
```

Quick sort :- (Faster)

→ extra memory memory consumption is less,

int partition (int arr[], int low, int high)

```
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] <= pivot)
        {
            i++;
            swap (arr[i], arr[j]);
        }
    }
    swap (arr[i + 1], arr[high]);
    return (i + 1);
}
```

void quicksort (int arr[], int low, int high)

```
{
    if (low < high)
    {
        int p = partition (arr, low, high);
        quicksort (arr, low, p - 1);
        quicksort (arr, p + 1, high);
    }
}
```

quick_sort (int *arr, int low, int high)

```
{
    if (low < high)
    {
        int index = partition(arr, low, high);
        quick_sort(arr, low, index - 1);
        quick_sort(arr, index + 1, high);
    }
}
```

Hashing :-

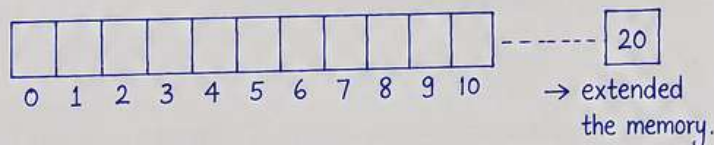
Key is present in the index (location) known by us. It will store the data in the known index.

Why we use hashing means :-

→ insert, delete, search, $O(1)$ → best
and
 $O(n)$ → worst.

Components :-

1. Hash table :-



2. Hash function :- It will provide the index in which the given data (key) should be stored.

Hashing function :-

$$h(x) = \text{data} \% \text{size}$$

data	index
4	4
10	0
20	0 → (collision: index)

Time then collision will be access same index multiple.

→ To overcome the collision technique we use collision resolution technique.

→ open address { Linear probing
→ direct address chain. { Quadratic probing

Linear probing :-

It will provide the new index for the data.

1. Linear probing :-

$$h(x) = (\text{data} \% \text{size}) + i \rightarrow i = 1$$

$$h(x) = (10 \% 5) + 1 \Rightarrow 1$$

$$h(x) = (30 \% 5) + 1 \Rightarrow 1 \text{ (collision)} \quad i = 1$$

$$= (30 \% 5) + 2 \Rightarrow 2 \text{ (collision)} \quad i = 2$$

$$= (30 \% 5) + 3 \Rightarrow 3 \quad i = 3$$

Quadratic probing :- Increment is

$$h(x) = (\text{data} \% \text{size}) + i^2$$

Direct chain :-

Large amount of data we need to search, so for the hashing.

0	1	2	3	4	→ key
-1	-1	-1	-1	-1	↔ value
NULL	NULL	NULL	NULL	NULL	↔ link

10
10
NULL

→ If in case demand will come we use this.

```
typedef struct hash_table {
    int key;
    int value;
    struct hash_table *link;
} hash_t;
```

```
int main() {
    hash_t arr[size]; // array of structure
    create_hash(arr, size);
}
```

```
create_hash(hash_t *arr, int size)
{
    for (int i = 0; i < size; i++)
    {
        arr[i].key = i;
        arr[i].value = -1;
        arr[i].link = NULL;
    }
}
```

```

arr[i].value = -1;    arr[i+1].value = -1;
arr[i].link = NULL;  arr[i+1].link = NULL;
}
}

```

insert_ht (hash_t *arr, int size, int data)

```

{
    int key = data % size; // To find out the index.
    if (arr[key].value == -1)
    {
        arr[key].value = data;
        return success;
    }
    else if (arr[key].value != -1 && arr[key].link == NULL)
    {
        // creating 1st node sub in hash.
        hash_t *new = malloc(size of (hash_t));
        new->key = key;
        new->value = data;
        new->link = NULL;
        arr[key].link = new;
        return success;
    }
    else if (arr[key].value != -1 && arr[key].link != NULL)
    {
        // create sub node in hash.
        hash_t *new = malloc(size of (hash_t));
        new->key = key;
        new->value = data;
        new->link = arr[key].link;
        arr[key].link = new;
        return success;
    }
    else
    {
        // In case, use give negative value for this, we have
        // to keep condition.
        if (data > 0)
            data % 5;
        else
            -(data) % 5;
    }
}

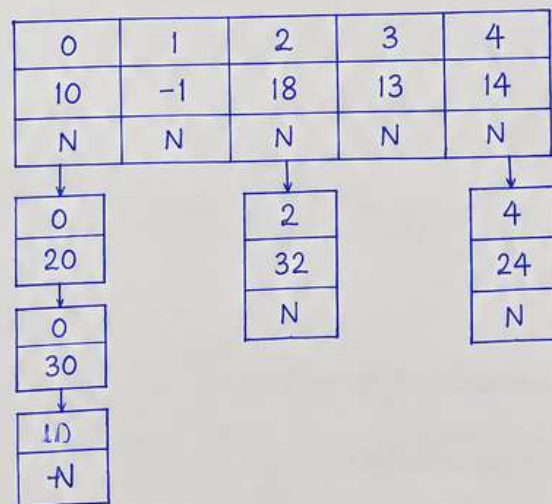
```

Search_hashtable (hash_t *arr, int size, int target)

```

{
    key = data % size;
    if (arr[key].value == -1)
    {
        return data_not_found;
    }
    if (arr[key].value == target)
    {
        return success;
    }
    else
    {
        hash_t *temp = arr[key].link;
        while (temp != NULL)
        {
            if (temp->value == target)
            {
                return success;
            }
            else
            {
                temp = temp->link;
            }
        }
        return data_not_found;
    }
}

```



Delete_hashtable (hash_t *arr, int size, int data)

```

{
    key = data % size;
    if (arr[key].value == -1)
    {
        return failure;
    }
    if (arr[key].value == data && arr[key].link == NULL)
    {
        arr[key].value = -1;
        return success;
    }
    else if (arr[key].value == data && arr[key].link != NULL)
    {

```

else

{

*temp = arr[key].link;

while (temp != NULL)

{

if (temp->value == data)

{

if (prev == NULL)

{

arr[key].link = temp->link;

free(temp);

}

else

{

prev->link = temp->link;

free(temp);

}

else

{

prev = temp;

temp = temp->link;

}

}

}

destroy :-

destroy_int (hash_t *arr, int size)

{

for (i = 0 ; i < size ; i++)

{

if (arr[i].value != -1)

{

arr[i].value = -1;

hash_t *temp;

while (arr[i].link != NULL)

{

temp = arr[i].link;

arr[i].link = temp->link;

free(temp);

}

}

}

return success;

}

TREES :- It having node.

→ It is a non-linear data structure.

→ add data not depend on sequential manner.

Why we go for trees :-

→ Insertion and deletion is simple.

→ Time complexity best $O(1)$,
worst average $\rightarrow O(\log n)$

Disadvantage :-

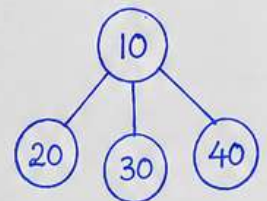
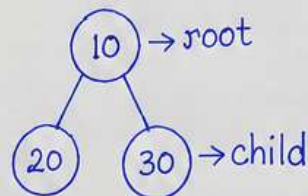
→ duplicate's are not allowed. (only Binary search)

→ It will store the data technical way.

Types of trees :-

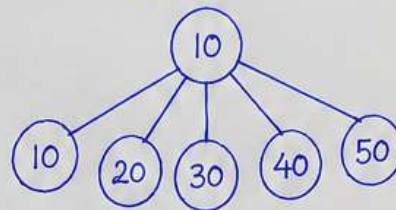
1. Binary trees

2. ternary tree :- It has 3 childs



3. n - ary tree :- depends upon user input - child will be created.

n = 5.



4. self - balancing tree :-

1. Red - Black tree ($O(\log n)$ only 2 rotations required.

2. AVL (height $O(\log n)$ we don't have many rotations. They take $O(\log n)$).

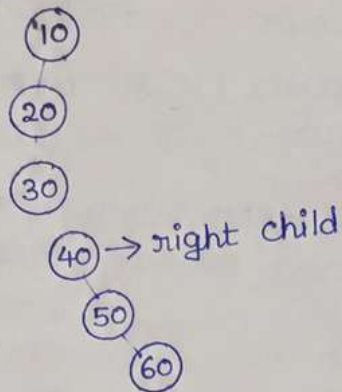
5. degenerative tree :-

They are two types :-

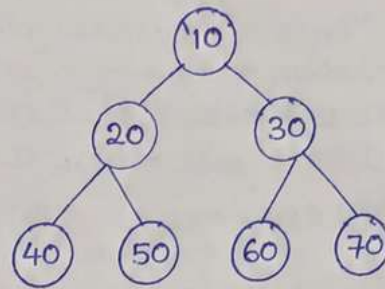
1. Left tree.

2. right tree.

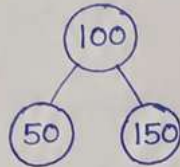
Left tree :-



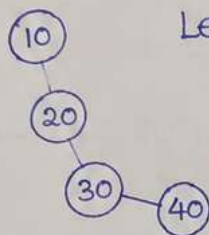
(6) complete - binary tree :-



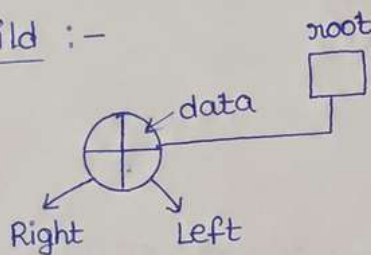
Binary search :- The child part one side low & high.



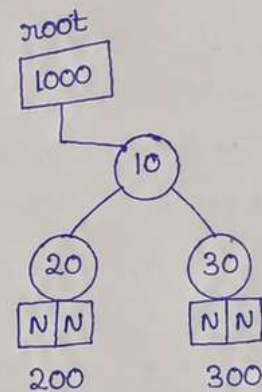
Pathology :- we can create either right child & left child.



Child :-



eg:-



```

Struct Tree {
    int data;
    Struct Tree *left;
    Struct Tree *right;
}; Trees;
    
```

```

Int main () {
    Tree_t *root = NULL;
    Insert_Tree (&root, 50)
}
    
```

Search_Tree (Tree_t *root, int target)

```

{
    If (root != NULL)
    {
        If (root->data == target)
            return success;
        else if (root->data < target)
            return Search_Tree(root->right, target);
        else if (root->data > target)
            return Search_Tree(root->left, target);
    }
}
    
```

Insert_Tree (Tree **root, int data)

```

{
    Tree *new = malloc (sizeof(Tree));
    new->data = data;
    new->left = new->right = NULL;
    If (*root == NULL)
    {
        *root = new;
        return success;
    }
    else
    {
        Tree_t *temp = *root, *prev = NULL;
        while (temp != NULL)
        {
            If (data < temp->data)
            {
                prev = temp;
                temp = temp->left;
            }
            else
            {
                temp = temp->right;
            }
        }
        If (data < prev->data)
            prev->left = new;
        else
            prev->right = new;
    }
}
    
```

```

else {
    *temp = root → right, *prev = NULL;
    while (temp != NULL) {
        prev = temp;
        temp = temp → link;
        root → data = temp → data;
    }
    prev → left = left → right;
    free(temp);
}
else {
    return failure;
}

```

Delete element :-

```

delete_t * delete_element (Tree_t * root, int target)
{
    if (root == NULL)
    {
        return data_not_found;
    }
    if (target < root → data)
        root → left = delete_element (root → left, target);
    else if (target > root → data)
        root → right = delete_element (root → right, target);
    else {
        data_found;
        1. delete left node;
        if (root → right == NULL && root → left == NULL) {
            free(root);
            root = NULL;
            return root;
        }
        else if (root → left != NULL && root → right == NULL) {
            // root only have left child
            *temp = root → left;
            free(root);
            return temp;
        }
        else if (root → left == NULL && root → right != NULL) {
            // root only have right child
            *temp = root → right;
            free(root);
            return temp;
        }
    }
}

```

Stack application :-

eg:- $3 + 3 \mid 4 * 6 + (2 - 1)$

→ to evaluate the expression we go for the application

1. Infix → a + b;	$\left. \begin{array}{l} \text{Infix} \rightarrow \text{Prefix} + \text{Postfix} \rightarrow O(n) \\ \text{Evaluation of prefix/postfix} \rightarrow O(n) \end{array} \right\} O(n)$
2. prefix → ab ++ ab	
3. postfix → ab ++	

Infix :- $2 * 3 - 3 + 8 / 4 (1 + 1)$

$2 * 3 - 3 + 8 / 4 (1 + 1)$

$2 * 3 - 3 + 8 / 4 (2)$

$2 * 3 - 3 + 8 / 4 / 2$

$2 * 3 - 3 + 84 / 11 +$

$23 * 3 - 84 / 11 + /$

$23 * 3 - 84 / 11 + / + \rightarrow \text{Postfix}$

Infix - postfix conversion

```
stack_t *st) {
    int i = 0, j = 0;
    while (infix[i] != '\0') {
        if (infix[i] == '0' && infix[i] <= '9') {
            postfix[j++] = infix[i++];
        }
        else if (infix[i] == '(') {
            push(st, infix[i]);
        }
        else if (infix[i] == ')') {
            while (peek(st) != '(') {
                postfix[j++] = peek(st);
                pop(st);
            }
            pop(st); // remove the '('
        }
        else { // operator
            while (st > top != -1 && priority(infix[i]) <= priority(peek(st))) {
                postfix[j++] = peek(st);
                pop(st);
            }
            push(st, infix[i]);
            i++;
        }
    }
    while (st > top != -1) { // pop all remaining operators
        postfix[j++] = peek(st);
        pop(st);
    }
    return success;
}
```

Rules for the prefix conversion :-

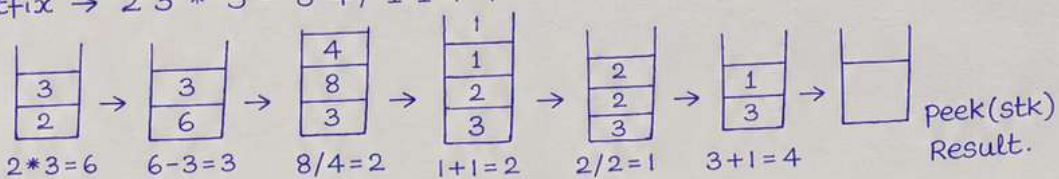
1. If the digit occurs directly store in the prefix array.
2. Whenever the '(' occurs, push it in the stack.
3. Whenever the ')' occurred, pop all the elements from the stack until reaches '(', store all the popped operator's in the prefix array.
4. operator's : (+, -, *, /)
 - 4.1 : check the stack is empty, if yes push it.
 - 4.2 : check the stack is non empty, check the priority(infix[i]) and priority(peek(stack)).

Prefix Evaluation :-

1. If the character prefix[i] is a digit, push it in the stack.
2. If operator pop two operands from stack and do operation.
 $\langle op1 \rangle \text{ operator } \langle op2 \rangle$ (op1 popped first, op2 next)
Result $\rightarrow$ should push in the stack.

Infix $\rightarrow 2 * 3 - 3 + 8 / 4 (1 + 1)$

Postfix $\rightarrow 2 3 * 3 - 8 4 / 1 1 + +$



Infix to Prefix conversion :-

1. If digit, store it in prefix.
2. If '(' push it in stack.
3. If ')' pop all the elements from stack until '(' occurs, store all the popped operator's in prefix.